

Smart Intrusion Detection & Threat Monitoring System (SIDTMS)

A report Submitted in partial fulfillment of the requirements for the award of degree of

Batchelor of Technology

Computer Science Engineering

Submitted to

LOVELY PROFESSIONAL UNIVERSITY

PHAGWARA, PUNJAB



L OVELY
P ROFESSIONAL
U NIVERSITY

From February 2026 to March 2026

SUBMITTED BY

Name – Ankit Kumar

Registration number – 12318018

Signature of the Student :

To whom so ever it may concern

I, Ankit Kumar ,12318018 , hereby declare that the work done by me on “ Smart Intrusion Detection & Threat Monitoring System (SIDTMS)” from February 2026 to March 2026, is a record of original work for the partial fulfillment of the requirements for the award of the degree.

B. TECH CSE

Ankit Kumar (12318018)

Signature of the student:
Dated: 23/03/2026

Abstract

This paper presents a lightweight intrusion detection and threat monitoring system designed for real-time network analysis. The system captures packets using Python-based tools and applies rule-based detection to identify suspicious activities such as port scanning and abnormal traffic patterns. Detected threats are logged and visualized through a web-based dashboard. The proposed system provides a low-cost alternative to traditional Security Information and Event Management solutions.

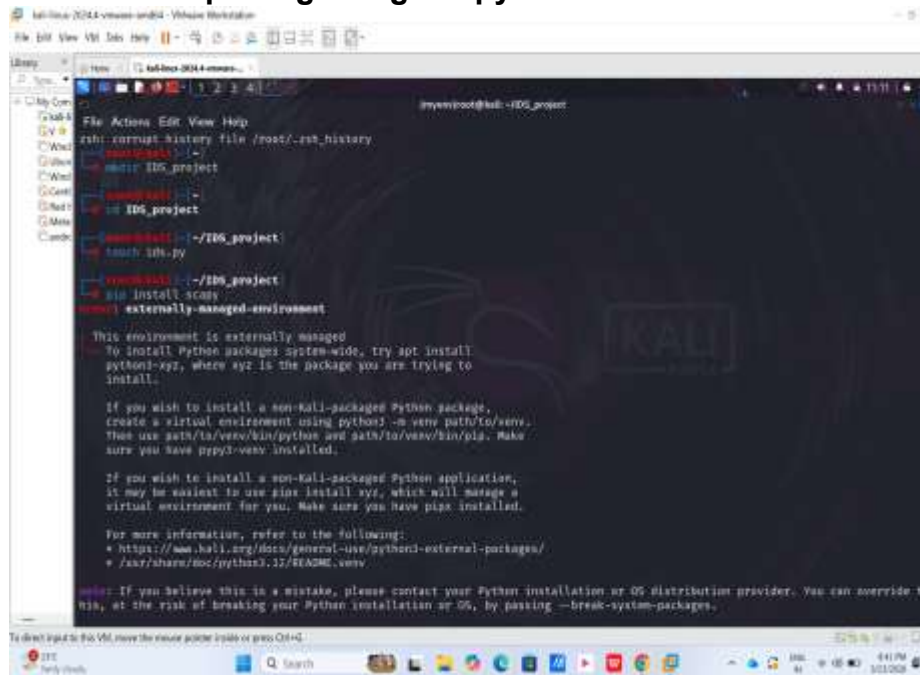
Keywords— Intrusion Detection System, Cybersecurity, Network Monitoring, Scapy, Flask

I. Introduction

With the increasing dependence on digital systems, cyber threats have become a major concern. Intrusion Detection Systems are essential tools that help in identifying malicious activities within a network. This project focuses on developing a simple and efficient IDS using Python technologies to monitor network traffic and detect anomalies.

II. Methodology

1. Packet capturing using Scapy



```
root@kali: ~/IDS_project
rsh: corrupt history file /root/.rsh_history
root@kali: ~
root@kali: ~# mkdir IDS_project
root@kali: ~# cd IDS_project
root@kali: ~/IDS_project
root@kali: ~/IDS_project# touch idn.py
root@kali: ~/IDS_project# pip install scapy
WARNING: externally-managed-environment

This environment is externally managed
To install Python packages system-wide, try apt install
python3-xyz, where xyz is the package you are trying to
install.

If you wish to install a non-Kali-packaged Python package,
create a virtual environment using python3 -m venv path/to/venv.
Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
sure you have pipx installed.

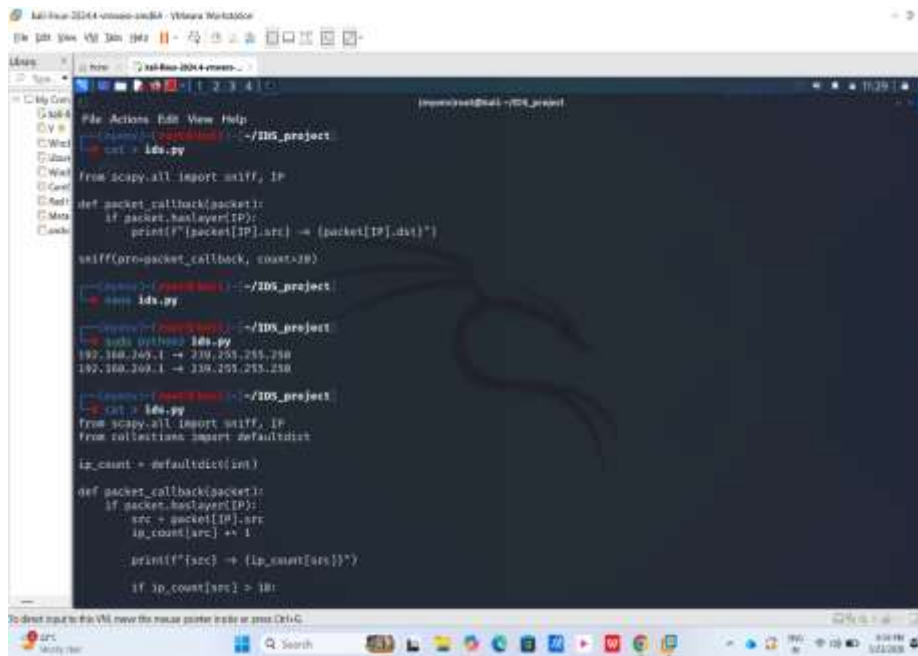
If you wish to install a non-Kali-packaged Python application,
it may be easiest to use pipx install xyz, which will manage a
virtual environment for you. Make sure you have pipx installed.

For more information, refer to the following:
• https://www.kali.org/docs/general-use/python3-external-packages/
• /usr/share/doc/python3.12/README.venv

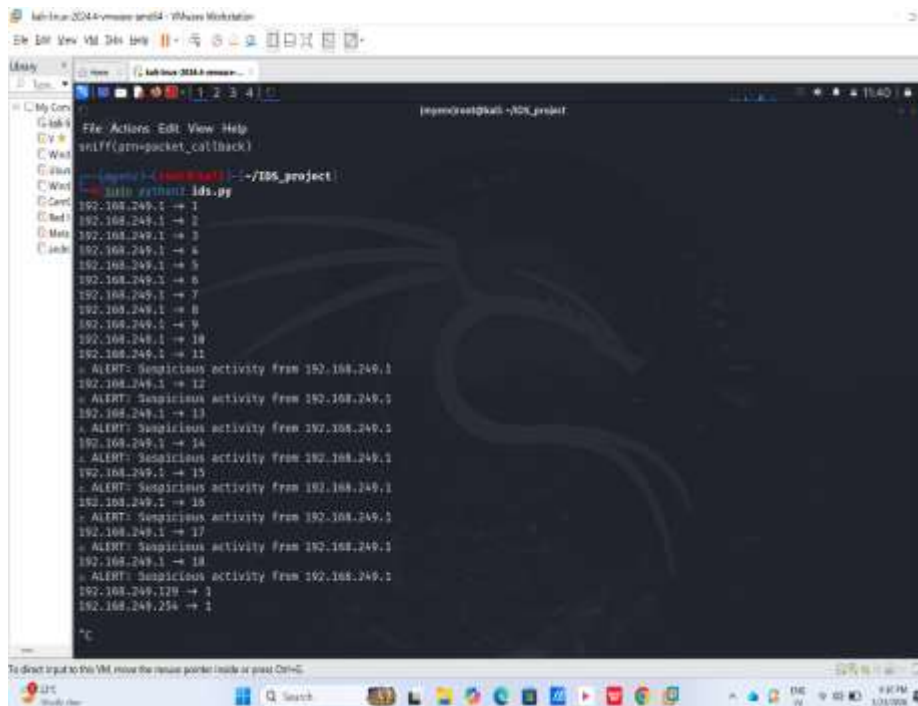
WARNING: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You can override t
his, at the risk of breaking your Python installation or OS, by passing --break-system-packages.
To direct output to the VML, use the venv option inside or pass OSG=1.
```

Fig.1: Project Setup in Kali Linux

2. Extraction of IP and TCP information



3. Application of rule-based detection logic



[illegible]


```
kali-linux-2024-secure-and04 - VMware Workstation
File Edit View VM Tools Help
cat: log: No such file or directory
root@kali:~/IDS_project# cat alerts.log
2026-03-23 11:43:19.959971 - 192.168.249.1 suspicious
2026-03-23 11:43:20.863767 - 192.168.249.1 suspicious
2026-03-23 11:43:20.964758 - 192.168.249.1 suspicious
2026-03-23 11:43:20.855109 - 192.168.249.1 suspicious
2026-03-23 11:43:20.138487 - 192.168.249.1 suspicious
2026-03-23 11:43:20.439704 - 192.168.249.1 suspicious
2026-03-23 11:43:20.688467 - 192.168.249.1 suspicious
2026-03-23 11:43:20.289958 - 192.168.249.1 suspicious
2026-03-23 11:43:20.301983 - 192.168.249.1 suspicious
2026-03-23 11:43:20.282628 - 192.168.249.1 suspicious
2026-03-23 11:43:20.336613 - 192.168.249.1 suspicious
2026-03-23 11:43:20.429157 - 192.168.249.1 suspicious
2026-03-23 11:43:20.439704 - 192.168.249.1 suspicious
2026-03-23 11:43:20.688467 - 192.168.249.1 suspicious
2026-03-23 11:43:20.584281 - 192.168.249.1 suspicious
2026-03-23 11:43:20.531648 - 192.168.249.1 suspicious
2026-03-23 11:43:20.727416 - 192.168.249.1 suspicious
2026-03-23 11:43:20.742233 - 192.168.249.1 suspicious
2026-03-23 11:43:20.776853 - 192.168.249.1 suspicious
2026-03-23 11:43:20.807567 - 192.168.249.1 suspicious
2026-03-23 11:43:20.898881 - 192.168.249.1 suspicious
2026-03-23 11:43:21.138184 - 192.168.249.1 suspicious
2026-03-23 11:43:21.196402 - 192.168.249.1 suspicious
2026-03-23 11:43:21.211502 - 192.168.249.1 suspicious
2026-03-23 11:43:21.258856 - 192.168.249.1 suspicious
2026-03-23 11:43:21.277213 - 192.168.249.1 suspicious
2026-03-23 11:43:21.278488 - 192.168.249.1 suspicious
2026-03-23 11:43:21.287928 - 192.168.249.1 suspicious
2026-03-23 11:43:21.389331 - 192.168.249.1 suspicious
2026-03-23 11:43:21.416181 - 192.168.249.1 suspicious
```

```
kali-linux-2024-secure-and04 - VMware Workstation
File Edit View VM Tools Help
root@kali:~/IDS_project# cat ids.py
from scapy.all import sniff, IP, TCP
from collections import defaultdict

port_scan = defaultdict(set)

def packet_callback(packet):
    if packet.haslayer(IP) and packet.haslayer(TCP):
        src = packet[IP].src
        port = packet[TCP].dport

        port_scan[src].add(port)

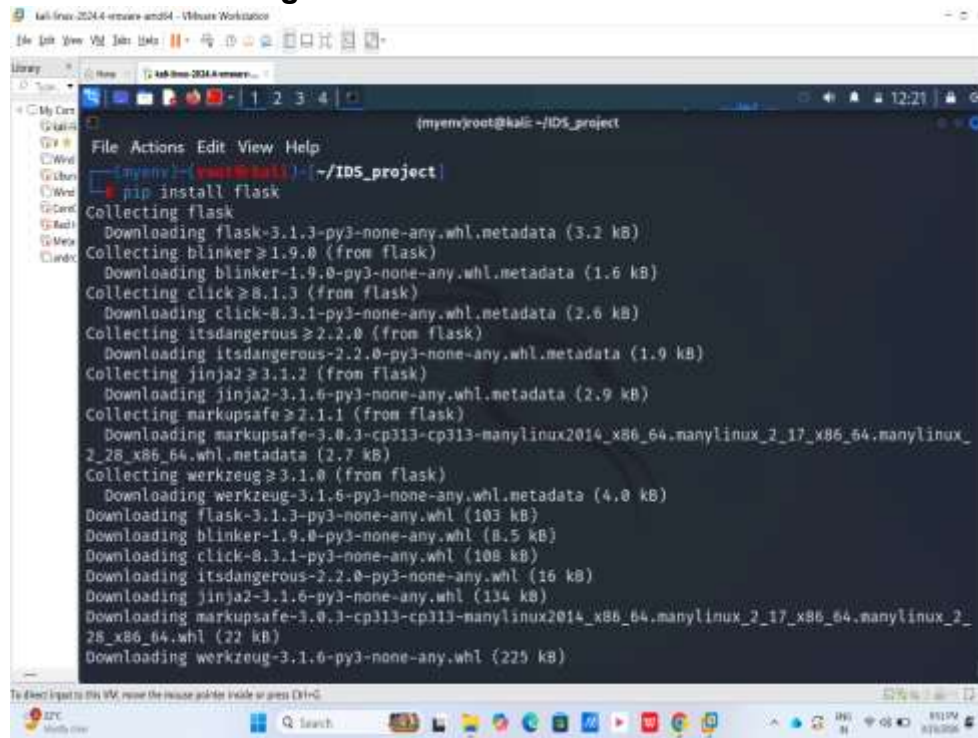
        if len(port_scan[src]) > 10:
            print(f"ALERT: Port scanning from {src}")

sniff(prn=packet_callback)

root@kali:~/IDS_project# nmap -p 1-100 localhost
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-23 13:47:00
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000007s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 99 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
Nmap done: 1 IP address (1 host up) scanned in 0.14 seconds

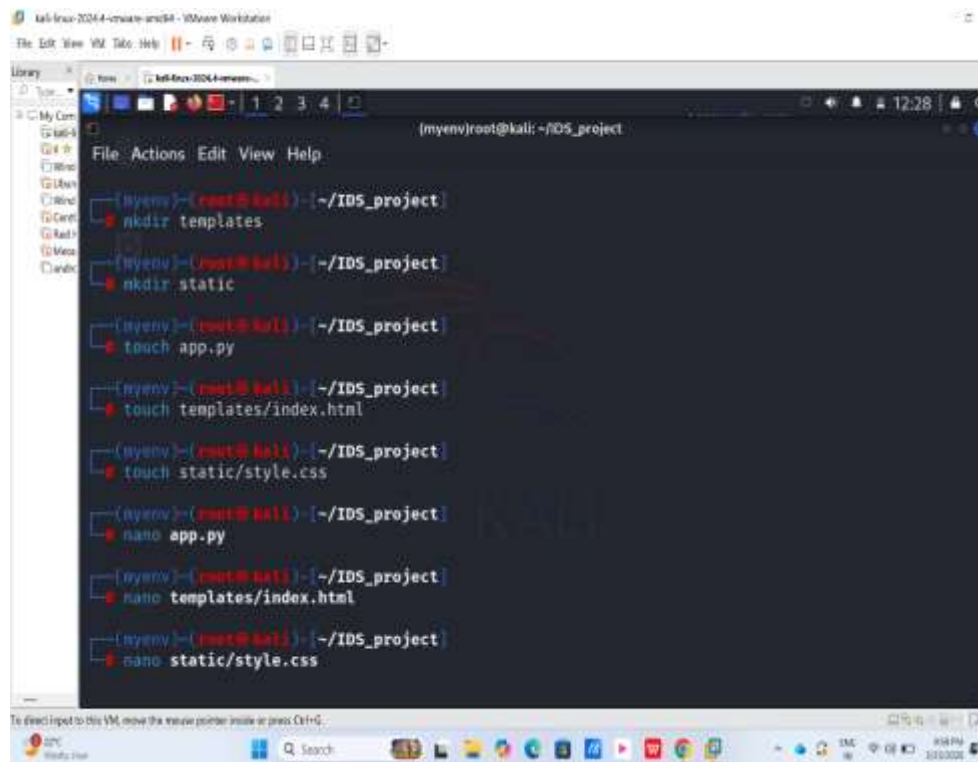
root@kali:~/IDS_project# sudo python3 ids.py
```


5. Visualization using Flask dashboard



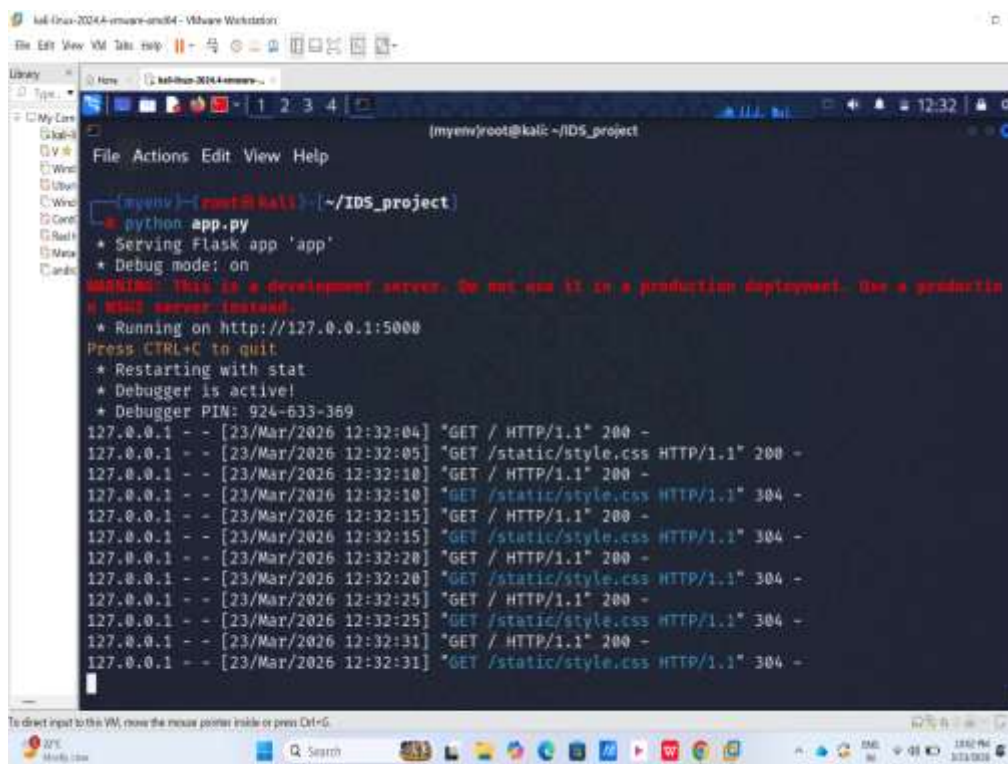
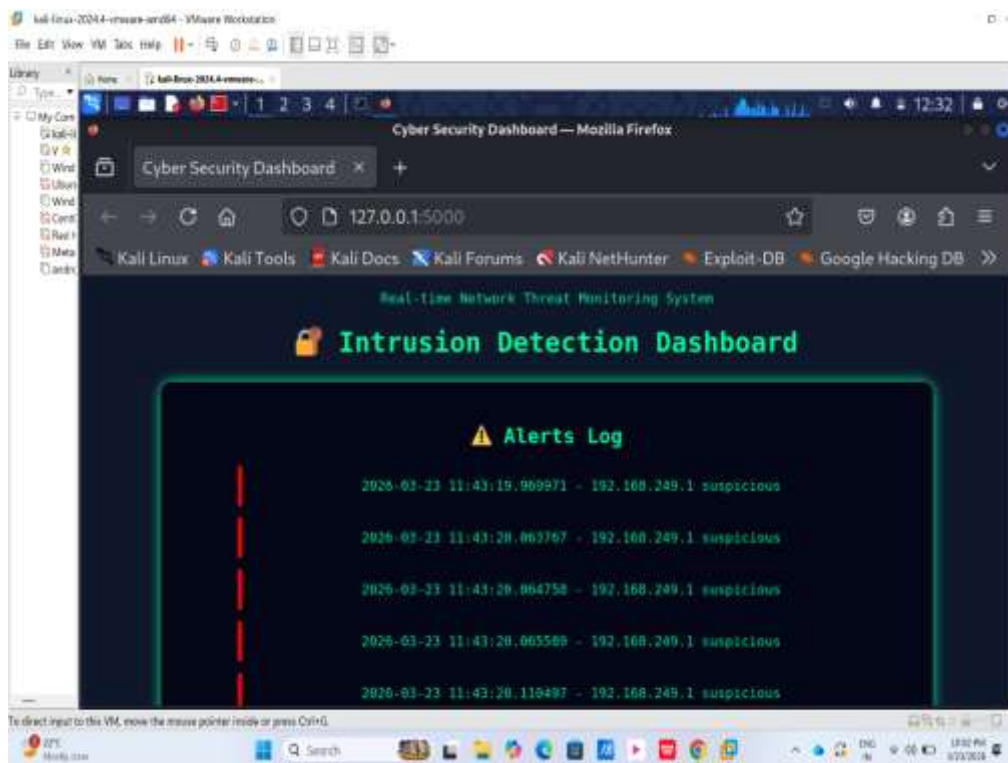
The screenshot shows a terminal window titled "kali-linux-2024-4-vmware-amd64 - VMware Workstation". The user is in the directory ~/IDS_project. The command `pip install flask` has been executed, and the output shows the collection and downloading of various dependencies including flask, blinker, click, itsdangerous, jinja2, markupsafe, and werkzeug. The terminal output is as follows:

```
(myenv)root@kali: ~/IDS_project
File Actions Edit View Help
(myenv)~(root@kali)~|~/IDS_project|
$ pip install flask
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.6-py3-none-any.whl.metadata (4.0 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.1-py3-none-any.whl (108 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
Downloading werkzeug-3.1.6-py3-none-any.whl (229 kB)
```



The screenshot shows the same terminal window as above, but now the user is creating static files for the Flask application. The commands and their outputs are as follows:

```
(myenv)root@kali: ~/IDS_project
File Actions Edit View Help
(myenv)~(root@kali)~|~/IDS_project|
$ mkdir templates
(myenv)~(root@kali)~|~/IDS_project|
$ mkdir static
(myenv)~(root@kali)~|~/IDS_project|
$ touch app.py
(myenv)~(root@kali)~|~/IDS_project|
$ touch templates/index.html
(myenv)~(root@kali)~|~/IDS_project|
$ touch static/style.css
(myenv)~(root@kali)~|~/IDS_project|
$ nano app.py
(myenv)~(root@kali)~|~/IDS_project|
$ nano templates/index.html
(myenv)~(root@kali)~|~/IDS_project|
$ nano static/style.css
```



III. System Architecture

The system consists of four major components: packet sniffer, detection engine, logging system, and web dashboard. Network traffic is captured and analyzed to detect threats, which are then logged and displayed on the dashboard.

IV. Results

The system successfully captures network packets and detects suspicious activities such as port scanning. Logs are generated and displayed on the dashboard, demonstrating real-time monitoring capabilities.

V. Conclusion

The developed IDS provides an effective and low-cost solution for monitoring network security. It demonstrates how basic cybersecurity mechanisms can be implemented using Python. Future enhancements may include machine learning-based detection and cloud integration.

References

- 1 Python Software Foundation, Python Documentation.
- 2 Scapy Documentation.
- 3 Flask Web Framework Documentation.

